

בלאק-ג'ק רב-משתתפים

משחק קלפים מאובטח בארכיטקטורת לקוח-שרת

בלאק-ג'ק רב-משתתפים	שם העבודה:
_____	שם התלמיד:
_____	ת.ז. התלמיד:
_____	שם המנחה:
_____	שם בית הספר:
הגנת סייבר ומערכות הפעלה	שם החלופה:
_____	תאריך הגשה:

חלופת התמחות: הגנת סייבר ומערכות הפעלה

מסלול: הנדסת תוכנה 883589

עמוד שער

“ הערה למילוי: את עמוד השער יש לעצב במסמך הוורד הסופי, ולכלול בו את הפרטים הבאים. השאירו מקום ללוגו בית הספר בראש הדף.

- לוגו בית הספר: _____
- שם בית הספר: _____
- שם העבודה: בלאק-ג'ק רב-משתתפים - משחק קלפים מאובטח בארכיטקטורת לקוח-שרת
 - שם התלמיד: _____
 - ת.ז. התלמיד: _____
 - שם המנחה: _____
- שם החלופה: הגנת סייבר ומערכות הפעלה
 - תאריך ההגשה: _____

תוכן עניינים

“ בעת המרת המסמך לוורד / Google Docs יש לייצר תוכן עניינים אוטומטי על בסיס הכותרות (Heading 1, Heading 2, Heading 3) שבמסמך

- מבוא
- מבנה / ארכיטקטורה של הפרויקט
 - מימוש הפרויקט
 - מדריך למשתמש
- סיכום אישי / רפלקציה
 - ביבליוגרפיה
 - נספחים

1. מבוא

1.1 ייזום

1.1.1 תיאור ראשוני של המערכת

הפרויקט הוא משחק בלאק-ג'ק (21) רב-משתתפים המופעל בארכיטקטורת לקוח-שרת על גבי רשת. מספר שחקנים מתחברים מהמחשבים שלהם לשרת מרכזי, יושבים סביב שולחן וירטואלי משותף ומשחקים זה לצד זה כנגד דילר ממוחשב, בדיוק כמו בקזינו אמיתי.

המוצר המוגמר כולל:

- שרת משחק הרץ ברקע, מקבל חיבורים, מנהל את חוקי המשחק, את הקופה ואת פרופילי השחקנים, ושומר את הנתונים על הדיסק.
- לקוח גרפי (Tkinter) המוצג על מסך השחקן עם מסך התחברות / הרשמה, שולחן בלאק-ג'ק עם הקלפים, כפתורי פעולה (Hit / Stand / Double), שדה הימור, סרגל פרופיל ומסך צ'אט.
- שכבת אבטחה - לחיצת יד מוצפנת בין הלקוח לשרת והצפנה סימטרית של כל הודעה לאחר מכן, כך שאף מאזין ברשת לא יוכל לקרוא את התוכן או לשלוח הודעות מזויפות.
- שכבת הרשאות - משתמשים נרשמים עם שם משתמש וסיסמה, סיסמאותיהם נשמרות בצורה מגובבת בלבד, וכל פעולה במשחק מחייבת הזדהות מול השרת.

מדוע בחרתי בפרויקט זה: בלאק-ג'ק הוא משחק מספיק פשוט בכללים שלו כדי שהמיקוד העיקרי יישאר בנושאים של החלופה - תקשורת, ריבוי לקוחות, שמירת מצב והגנת סייבר - אך מספיק עשיר כדי לדרוש מכונת מצבים אמיתית, סנכרון בין מספר שחקנים ולוגיקה תלוית כללים (אסים, הכפלה, פוש, בלאק-ג'ק טבעי). הוא גם מספק תרחיש שבו האבטחה אינה דקורטיבית: אם ההצפנה לא תעבוד, שחקן יוכל לזייף קלפים או לגנוב את הקרדיטים של חבריו לשולחן.

אתגרים שצפיתי לפני התחלת הפרויקט:

- תכנון פרוטוקול תקשורת בינארי / טקסטואלי שמתמודד עם הזרם הבלתי מוגדר של TCP.
- ניהול ריבוי לקוחות במקביל בעזרת תהליכונים (threads) מבלי ליצור תנאי מירוץ (race conditions).
- יישום נכון של לחיצת יד הצפנה אסימטרית-סימטרית ושל גיבוב סיסמאות.
- בנייה של מכונת מצבים נכונה למשחק כך שלא ניתן יהיה לבצע פעולה מחוץ לתור.
- יצירת ממשק משתמש שיציג את אותו "שולחן" באופן עקבי לכל השחקנים.

1.1.2 הגדרת הלקוח

המערכת מיועדת לשני קהלי יעד:

- משתמשי קצה - נערים ומבוגרים המחפשים משחק קלפים חברתי, נטול תשלום אמיתי, שאפשר לשחק בו עם חברים על אותה רשת מקומית.
- מורים ותלמידים בחלופת הגנת הסייבר - המערכת משמשת גם להדגמה חינוכית של עקרונות תקשורת מאובטחת, ריבוי לקוחות, אחסון מאובטח של פרטי משתמשים והפרדת לקוח-שרת.

1.1.3 הגדרת יעדים ומטרות

- לאפשר לכמה שחקנים לשחק בלאק-ג'ק בו-זמנית סביב שולחן וירטואלי אחד.
- לאפשר לכל שחקן להירשם, להתחבר ולשמור פרופיל קבוע (קרדיטים, ניצחונות, הפסדים, יחס W/L).
- להבטיח שכל התקשורת בין הלקוח לשרת מוצפנת ומאומתת.
- למנוע מצב שבו לקוח זדוני ישפיע על מצב המשחק של שחקנים אחרים או יזייף את היתרה שלו.
- להציג ממשק משתמש ברור ונעים שמרגיש דומה לשולחן בלאק-ג'ק אמיתי.
- לשמור על נתוני המשתמשים בצורה מתמשכת בין הפעלות של השרת.

1.1.4 בעיות, תועלות וחסכוניות

- הבעיה שאנחנו פותרים: משחקי קלפים מקוונים מעטים מתאימים להדגמת עקרונות סייבר; רובם או "קופסה שחורה" סגורה או דורשים תשתית ענן יקרה.
- תועלות צפויות:
 - חוויית משחק חברתית מקומית, ללא צורך בהרשמה לשירות חיצוני.
 - סביבת לימוד אותנטית להבנת הצפנה, threads ו-sockets.
 - פרופילי משתמש מתמשכים, כך שניתן לעקוב אחרי ההתקדמות לאורך זמן.
- שירותים שהמערכת מספקת: הרשמה / התחברות, הימור, חלוקת קלפים, פעולות שחקן (Hit / משחק דילר, חישוב תוצאות ותשלום אוטומטי, צ'אט בין שחקנים, רענון, Stand / Double) מצב שולחן בזמן אמת.

1.1.5 סקירת פתרונות קיימים

חסרונות מבחינתנו	יתרונות	מערכת קיימת
סגור לחלוטין, דורש תשלום, אין גישה לקו	עיצוב מקצועי, נתונים אמיתיים	Online Casino 888 / PokerStars כמו
חד-שחקן, ללא רשת, ללא אבטחה	קוד פתוח, פשוט להבנה	Open-source Blackjack ב-Python (CLI)
לא מדגים סוקטים גולמיים, מסתיר את שכבו	רב-משתתפים, נגיש	בדפדפ WebSocket Multiplayer Card Games

הפתרון שלי נבדל בכך שהוא רב-משתתפים אמיתי על TCP גולמי, עם לחיצת יד הצפנה שכתובה ידנית ועם שמירת פרופילים לא ב-DB חיצוני - כך שכל שכבה במערכת ניתנת לקריאה ולהבנה.

1.1.6 סקירת טכנולוגיית הפרויקט

הפרויקט נכתב כולו ב-Python 3.11. הספריות בהן נעזרתי הן ספריות סטנדרטיות (socket, threading, json, struct, secrets, hmac, hashlib, tkinter, dataclasses, enum, logging) למעט ספריית cryptography של Python Cryptographic Authority - ספריית הצפנה שעברה ביקורת ביטחונית והיא תקן דה-פקטו בעולם הפייתון.

אין כאן טכנולוגיה אקזוטית: TCP sockets, threads ו-Tkinter קיימים בפייתון מאז ומעולם. עם זאת, בחירה זו מקלה על קריאה והבנה של הקוד מבלי להסתבך עם פריימוורקים גדולים.

סייגים ומגבלות:

- המערכת תוכננה לרשת מקומית או לרשת אמינה; היא אינה מיועדת ל-internet פתוח (אין תעודה רשמית של CA, אין הגנת DDoS מתוככמת).
- דורש מערכת הפעלה עם תמיכה גרפית Tkinter מבוסס GUI.
- מספר השחקנים בשולחן מוגבל מעשית על ידי גודל המסך וקצב הרשת.

1.1.7 תיחום הפרויקט

בתחומים בהם הפרויקט עוסק:

- תקשורת מבוססת TCP sockets ובניית פרוטוקול מותאם אישית מעל זרם בתים.
- ריבוי לקוחות בעזרת threads.
- הצפנה היברידית (RSA-2048 + Fernet/AES-128) ולחיצת יד.
- גיבוב סיסמאות (PBKDF2-HMAC-SHA256 עם salt).
- שמירת מידע מתמשך בקובץ JSON עם כתיבה אטומית.
- בניית ממשק משתמש גרפי באירועים (Tkinter event loop).
- מכונת מצבים ניהולית למשחק רב-שלבי.

בתחומים בהם הפרויקט אינו עוסק:

- התשלומים האמיתיים, חיוב כרטיסי אשראי או הימורים בכסף.
- אחסון בענן / שרתים מבוזרים / load balancing.
- תאימות לדפדפן / WebSockets / מובייל נטיב.
- גרפיקה תלת-ממדית או אנימציות מורכבות של קלפים.

1.2 פירוט תיאור המערכת (אפיון)

1.2.1 תיאור מפורט של המערכת

המערכת בנויה משני יישומים נפרדים:

- שרת בלאק-ג'יק - יישום ללא ממשק (consoler / daemon). בעלייתו הראשונה הוא יוצר זוג מפתחות RSA, שומר אותם בקובץ server_rsa.pem, מאזין על פורט 5050, ומכין את קובץ הפרופילים profiles.json. מאותו רגע הוא מקבל חיבורים, פותח עבור כל אחד תהליכון נפרד, מנהל את כללי הבלאק-ג'יק ומשדר עדכונים לכל המחוברים.
- לקוח בלאק-ג'יק - יישום שולחני עם חלון Tkinter. בהפעלתו הוא מתחבר לשרת, מבצע לחיצת יד מוצפנת, ומציג למשתמש מסך התחברות. לאחר אימות נפתח מסך השולחן ובו לוחית הדילר, רשימת השחקנים הסביב, יד השחקן עצמו, סרגל פרופיל וכפתורי פעולה.

המערכת תומכת בריבוי שחקנים: כאשר שחקן נוסף מתחבר הוא מצטרף אוטומטית לסבב הבא ורואה את אותו שולחן.

1.2.2 פירוט היכולות לפי סוג משתמש

קיים סוג משתמש יחיד - שחקן (אין משתמש מנהל בנפרד; ניהול השרת מתבצע על ידי הפעלה / כיבוי שלו). היכולות שלו (ברמת כותרות):

- הרשמה למערכת (יצירת פרופיל חדש עם 10,000 קרדיטים).
- התחברות למערכת (אימות מול פרופיל קיים).
- הצטרפות לשולחן.
- הימור בכל סבב.
- ביצוע פעולות במשחק: Hit (קח עוד קלף), Stand (עצור), Double (הכפל הימור).
- צפייה בקלפי הדילר ובקלפים של שחקנים אחרים סביב השולחן.
- צפייה בפרופיל אישי ובסטטיסטיקה (קרדיטים, ניצחונות, הפסדים, יחס W/L).
- שיחה עם השחקנים האחרים בצ'אט.

1.2.3 תכנון בדיקות (קופסה שחורה)

בטבלה הבאה רשומות הבדיקות שתוכננו לפני המימוש; פירוט ההרצה והתוצאות נמצא בפרק "מימוש הפרויקט - חלק ג'".

אופן הביצוע המתוכנן	מטרת הבדיקה	#
בלקוח א': register("alice","secret123)	הרשמה תקינה של משתמש חדש	1
בלקוח א' להירשם עם "alice"; בלקוח ב'	מניעת רישום כפול	2
להזין סיסמה לא נכונה - יש לקבל הודעה כ	התחברות עם סיסמה שגויה	3
להתחבר עם "alice" פעמיים בו-זמנית -	מניעת התחברות כפולה לאותו משתמש	4
לחבר שניים, כל אחד מהמר, מבצע Hit/Sta	זרימת משחק מלאה לשני שחקנים	5
לבקש הימור הגדול מהקרדיטים - יש לקבל ש	הימור גבוה מהיתרה	6
בלקוח שאינו השחקן הפעיל ללחוץ Hit -	ניסיון פעולה מחוץ לתור	7
להריץ tcpdump על הפורט; חבילות לאחר	הצפנה - האזנה לרשת	8
לשלוח ידנית מסגרת בגודל < 64 KB - החי	חוסן מול הודעה ענקית	9
לעצור את השרת ולהפעיל שוב; להתחבר עם	שחזור פרופילים אחרי restart	10
שחקן מתנתק כשהוא הפעיל - התור עובר אוט	התנתקות באמצע סבב	11
לנסות להירשם עם רוחים / תווים מיוחדים	ולידציה של שם משתמש	12

1.2.4 לוח זמנים

משך בפועל	משך מתוכנן	אבן דרך	#
שבוע 2-1	שבוע 2-1	אפיון, לימוד sockets ו-threads	1
שבוע 3-5 (חרגה בשבוע אחד בגלל debug ש	שבוע 4-5	פרוטוקול תקשורת ולחיצת יד מוצפנת	2
שבוע 6-7	שבוע 6-5	מנוע משחק (Card / Deck / Hand / Table)	3
שבוע 7-8	שבוע 7	ניהול פרופילים והצפנת סיסמאות	4
שבוע 8-9	שבוע 8	שילוב שרת ולקוח, threads, broadcasts	5
שבוע 9-11	שבוע 10-9	GUI ב-Tkinter	6
שבוע 11-12	שבוע 11	בדיקות אינטגרציה ותיקון באגים	7
שבוע 12	שבוע 12	כתיבת תיק פרויקט	8

1.2.5 ניהול סיכונים

מה בוצע בפועל	תכנון להתמודדות	סיכון
הוטמעה RLock אחת ב-Table ועוד אחת	להגן על מצב המשחק בנעילה (Lock)	שני לקוחות threads בין Race conditions
החיבור נסגר בנימוס ומופיע לוג ברור; הל	לעטוף את ה-handshake ב-try/except ולסג	כשל בלחיצת היד או חוסר תאימות מפתחות
הוטמע ב-(_save_locked); אם השרת קורס	כתיבה אטומית temp + os.replace()	איבוד קובץ הפרופילים תוך כדי כתיבה (קר
הוטמע ב-protocol.py (MAX_MESSAGE_BY	מגבלת גודל 64 KB ובדיקת אורך לפני קר	לקוח זדוני ישלח הודעה ענקית
הוטמע ב-AppController.enqueue_messag	ה-network thread ידחוף הודעות לתור והת	thread-safe אינו Tkinter

1.3 תיאור תחום הידע - פירוט יכולות

1.3.1 יכולות בצד השרת

♦ יכולת: קבלת חיבור חדש וניהולו בתהליכון נפרד

- מהות: השרת מאזין על פורט קבוע. בכל פעם שלקוח מתחבר, נפתח עבורו ClientHandler נפרד שמטפל רק בו, כדי שמספר שחקנים יוכלו לפעול במקביל.
- פעולות נדרשות: socket.accept, threading.Thread, ניהול רשימת מטפלים פעילים, סגירה מסודרת בעת ניתוק.
- אובייקטים נחוצים: BlackjackServer, ClientHandler.

♦ יכולת: לחיצת יד הצפנה (RSA → Fernet)

- מהות: לאחר חיבור TCP מתבצעת חליפת מפתחות. השרת שולח לציבור את מפתחו הציבורי, הלקוח מגריל מפתח Fernet אקראי, מצפין אותו במפתח הציבורי ושולח חזרה. מהרגע הזה כל ההודעות מוצפנות סימטרית.
- פעולות נדרשות: טעינת/יצירת זוג RSA, סריאליזציה ל-PEM, פענוח OAEP, יצירת SecureChannel.
- אובייקטים נחוצים: RSAKeyPair, SecureChannel.

♦ יכולת: רישום משתמש חדש

- מהות: קליטת שם משתמש וסיסמה, ולידציה, גיבוב הסיסמה ושמירה בקובץ הפרופילים.
- פעולות נדרשות: ולידציית שם / סיסמה, PBKDF2 200,000 hash_password איטרציות + ללקוח auth_result הוספה לדיקט הפרופילים, שמירה אטומית, החזרת תשובה (salt).
- אובייקטים נחוצים: ProfileManager, UserProfile.

♦ יכולת: התחברות משתמש קיים

- מהות: השוואת הסיסמה הנכנסת מול הסיסמה המגובבת בזיכרון, באופן שעמיד מפני timing attacks.
- פעולות נדרשות: verify_password עם hmac.compare_digest; אם המשתמש אינו קיים - לבזז זמן דומה כדי לא לחשוף זאת; מניעת התחברות כפולה.
- אובייקטים נחוצים: ProfileManager, ClientHandler.

♦ יכולת: ניהול שולחן הבלאק-ג'ק

- מהות: מכונת מצבים שעוברת בין WAITING → BETTING → PLAYING → DEALER → RESULTS. בכל שלב מותר לבצע פעולות מסוימות בלבד.
- פעולות נדרשות: קבלת שחקן לשולחן, קבלת הימור, חלוקת קלפים, מעבר תור בין שחקנים, משחק הדילר, חישוב תוצאות, איפוס לסבב הבא.
- אובייקטים נחוצים: Table, Player, Dealer, Deck, Hand, Card, Phase.

♦ יכולת: שידור (broadcast) מצב השולחן

- מהות: לאחר כל אירוע משמעותי (חלוקה, פעולה, שינוי שלב) השרת בונה תמונת מצב נפרדת לכל שחקן (מסתיר את הקלף הנעלם של הדילר כל עוד אינו מוצג) ושולח לו אותה.
- פעולות נדרשות: סריאליזציה של Table ל-dict, החלפת הקלף הנעלם ב-placeholder, איטרציה על מטפלי הלקוחות.

- אובייקטים נחוצים: BlackjackServer, Table.snapshot_for, ClientHandler.send.

♦ יכולת: תשלום וזיכוי לפי תוצאת הסבב

- מהות: בסוף כל סבב מחושבות התוצאות, היתרות מתעדכנות וטבלת הניצחונות / הפסדים נשמרת לדיסק.
- פעולות נדרשות: השוואת ערכי ידיים, החלת תשלום (1:1, 3:2 לבלאק-ג'ק טבעי, החזר הימור בפוש), עדכון UserProfile, שמירה.
- אובייקטים נחוצים: Table._resolve_locked, ProfileManager.adjust_credits, record_result-.

1.3.2 יכולות בצד הלקוח

♦ יכולת: התחברות לשרת ולחיצת יד

- מהות: פתיחת socket, פענוח ה-server_hello, יצירת מפתח Fernet, הצפנתו במפתח הציבורי של השרת ושליחתו, אישור ready מוצפן.
- פעולות נדרשות: socket.connect, serialization.load_pem_public_key, SecureChannel.generate, encrypt_session_key.
- אובייקטים נחוצים: BlackjackClient, SecureChannel.

♦ יכולת: מסך התחברות / הרשמה

- מהות: הצגת שדות שם משתמש וסיסמה ושני כפתורים. שליחת login או register אל השרת והצגת התשובה.
- פעולות נדרשות: בניית fal, captureOf submit, ttk.Frame טיפול בשגיאה.
- אובייקטים נחוצים: LoginFrame, AppController.

♦ יכולת: מסך השולחן

- מהות: הצגת אזור הדילר, אזור השחקנים, סרגל פרופיל אישי, שדה הימור וכפתורי פעולה.
- פעולות נדרשות: ציור קלף ל-label, צביעת אדום לקלף לב/יהלום, סימון "(you)" לשחקן הנוכחי, איפשור / נטרול כפתורים לפי שלב.
- אובייקטים נחוצים: TableFrame.

♦ יכולת: רענון אסינכרוני של מצב השולחן

- מהות: הודעות מהרשת מגיעות בתהליכון רקע. הן נדחפות לתור, ו-GUI מנקה אותן ב-root.after(0,...) - כך שלא נוגעים בווידג'טים מתהליכון לא נכון.
- פעולות נדרשות: queue.Queue, callback, _drain_messages.
- אובייקטים נחוצים: AppController.

♦ יכולת: שליחת פעולה (Hit / Stand / Double / Bet / Chat)

- מהות: הקלקה של המשתמש מתורגמת להודעה מוצפנת לשרת.
- פעולות נדרשות: ולידציה לפני שליחה, BlackjackClient.send, טיפול בכשל רשת.
- אובייקטים נחוצים: TableFrame, AppController, BlackjackClient.

2. מבנה / ארכיטקטורה של הפרויקט

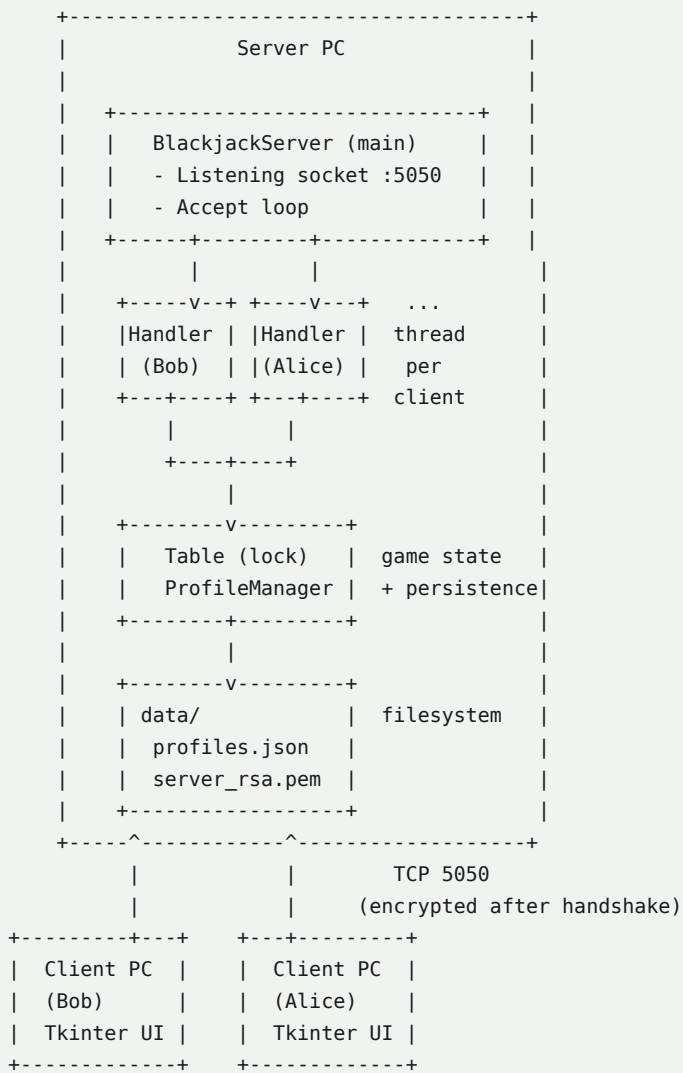
2.1 תיאור הארכיטקטורה של המערכת

המערכת בנויה בארכיטקטורת לקוח-שרת קלאסית עם שרת מרכזי יחיד. כל הקלפים, הקופה והכללים מנוהלים על ידי השרת. הלקוחות הם "משטחי תצוגה" שמציגים מצב ושולחים בקשות פעולה - הם לעולם אינם מקור האמת.

2.1.1 רכיבי החומרה והקשרים ביניהם

- מחשב שרת: מחשב יחיד עליו רץ `python -m blackjack.run_server`. דורש מערכת הפעלה (Linux / macOS / Windows), Python 3.11, מקום בדיסק לשני קבצים (`profiles.json`, `server_rsa.pem`).
- מחשבי לקוח: מחשב לכל שחקן, באותה רשת מקומית כמו השרת או ברשת אליה יש למחשב השרת `python -m blackjack.run_client` כל מחשב מריץ `port forwarding`.
 - רשת: TCP/IPv4 על פורט 5050 (ניתן לשינוי באמצעות משתני סביבה).

2.1.2 שרטוט ארכיטקטורה (חובה)



2.2 תיאור הטכנולוגיה הרלוונטית

סיבה לבחירה	טכנולוגיה	תחום
ברורה לקריאה, ספריית סטנדרט עשירה (soc)	Python 3.11	שפת תכנות
פיתוח רב-פלטפורמות; תמיכה במצב הרשאות אמין (ללא איבוד חבילות), פתוח לבניית פ	Linux / Windows / macOS	מ"ה (מערכת הפעלה)
תקן מקובל, מתאים להעברת מפתח חד-פעמי	גולמי, פורט 5050 TCP	תקשורת
מקנה גם סודיות וגם אימות שלמות	RSA-2048 + OAEP (cryptography library)	הצפנה אסימטרית
איטי בכוונה כדי להקשות brute-force	Fernet (AES-128-CBC + HMAC-SHA256)	הצפנה סימטרית
בנוי בתוך Python, ללא תלות חיצונית	איטרציות 200,000 PBKDF2-HMAC-SHA256, גיבוי סיסמאות	ממשק משתמש
פשוט מאוד, אין צורך ב-DB; עמיד בפני קר	Tkinter	אחסון
פשוט להבנה ומספיק יעיל בעומס משחק	קובץ JSON אחד, כתיבה אטומית עם os.rep	ריבוי לקוחות
	threading.Thread + threading.RLock	

2.3 תיאור זרימת המידע במערכת

2.3.1 זרימה כללית של חיבור שחקן

```

[Client]                                [Server]
| TCP connect :5050                      |
|----->|                               |
| server_hello {public_key_pem}          |
|<-----|                               |
| key_exchange {encrypted_session_key}   |
|----->|                               |
| ready (encrypted)                      |
|<-----|                               |
| login {username, password} (encrypted) |
|----->|                               |
|                                         | [authenticate]
| auth_result {ok, profile} (encrypted)  |
|<-----|                               |
| game_state {snapshot} (encrypted)      |
|<-----|

```

2.3.2 זרימת סבב יחיד

BETTING phase:

```

client --place_bet 200--> server --[debit profile]--> ProfileManager
server --[record bet]-----> Table
server --broadcast game_state-->all clients

```

(once all bets are in)

PLAYING phase:

```

Table.deal()      -> 2 cards each, dealer hidden card
server -> snapshot for each client
Loop over players:
  client (current) --action {hit|stand|double}--> server
  server --validates phase + turn--> Table.player_action()
  server -> broadcast game_state

```

DEALER phase:

```

server reveals dealer hole card
while dealer.should_hit(): dealer.add(deck.draw())
server -> broadcast

```

RESULTS phase:

```

for each player: compute outcome, payout, record W/L
server -> personal round_result + updated profile
Timer (6s) -> Table.prepare_next_round() -> back to BETTING

```

(שרטוט מלא ניתן להפיק ב-draw.io מתוך התרשים שלעיל; קבצי הפלואו צ'ארטים האינטראקטיביים flowchart.html ו-flowchart_deep.html במצורפים בנספחים)

2.4 תיאור האלגוריתמים המרכזיים

2.4.1 חישוב ערך יד בלאק-ג'יק עם אסים גמישים

הבעיה האלגוריתמית: בקלף אס ערכו 1 או 11. יש לבחור עבור כל יד את הצירוף המרבי שלא חוצה

אלגוריתמים קיימים:

- פתרון נאיבי: בדיקה רקורסיבית של כל הצירופים - $O(2^k)$ בכמות האסים.
- פתרון מקובל בקזינו: סופרים את כל האסים כ-1, ואז "משדרגים" אסים ל-11 כל עוד היד אינה חוצה 21.

הפתרון שנבחר: הפתרון השני, כי הוא $O(n)$ בלבד, פשוט להבנה ומכוסה בכל ספרי החוקים של בלאק-ג'יק. ראה `Hand.value` → `common/hand.py`.

2.4.2 ערבוב חבילת קלפים בצורה מאובטחת

הבעיה: יצירת סדר אקראי של 52×4 קלפים שלא ניתן לחיזוי על ידי תוקף.

אלגוריתמים קיימים:

- מהיר אך ניתן לחיזוי אם 624 ערכים, Mersenne Twister-משתמש ב- `random.shuffle`.
- רצופים נצפו.
- עם מקור אקראי קריפטוגרפי Fisher-Yates.

הפתרון שנבחר: Fisher-Yates עם `secrets.randbelow`. הספרייה `secrets` של Python בנויה במיוחד למטרות אבטחה ומשתמשת ב-`/dev/urandom` (או מקור OS מקביל). ראה `common/deck.py` → `Deck.resuffle`.

2.4.3 גיבוב סיסמאות PBKDF2

הבעיה: אחסון בטוח של סיסמאות כך שאם הקובץ ייגנב, התוקף לא יוכל לשחזר את הסיסמאות בקלות.

אלגוריתמים קיימים:

- Rainbow tables-פשוט - מהיר מדי, רגיש להתקפות מילון ו MD5 / SHA-1.
- אלפי פעמים; כל ניסיון ניחוש דורש אותו עבודה SHA256 חוזר על - salt עם PBKDF2.
- איטית.
- GPU/ASIC מודרניים יותר, חזקים גם נגד - Argon2 / scrypt / bcrypt.

הפתרון שנבחר: PBKDF2-HMAC-SHA256 עם 200,000 איטרציות ו-salt באורך 16 בתים, ייחודי לכל משתמש. נבחר משום שהוא חלק מספריית `hashlib` הסטנדרטית (אין תלות חיצונית) ועומד בהמלצות OWASP לרמת אבטחה סבירה. ראה `hash_password` → `common/crypto.py` → `verify_password`.

2.4.4 מסגור הודעות (framing) מעל TCP

הבעיה: TCP מספק זרם בתים, לא הודעות. הצד המקבל אינו יודע איפה מסתיימת הודעה אחת ומתחילה הבאה.

פתרונות קיימים: delimiter (כמו n ב-HTTP/IRC), אורך קבוע, או length-prefixed.

הפתרון שנבחר: length-prefix של 4 בתים (n, `struct.pack(">I", n)`). יתרונות: בינארי בטוח (יכול לשאת כל תוכן, כולל הצפנה), פשוט לפענוח, ניתן להגביל את הגודל המרבי לפני קריאה (64 KB בפרויקט) ובכך למנוע מתקפת מיצוי משאבים.

2.5 תיאור סביבת הפיתוח

- כלי הפיתוח:
 - Python 3.11 (CPython) (הרשמי)
 - עורך קוד: VS Code עם תוסף Python.
 - ניהול חבילות: pip.
 - חבילה חיצונית בודדת: cryptography.
 - לניהול גרסאות: git.
- כלי בדיקה:
 - הרצה ידנית של שרת ושני לקוחות מקבילים.
 - בדיקת end-to-end (כתובה ב-blackjack/test_e2e.py ב-Python) (בנספח).
 - לוידוא שהתעבורה מוצפנת tcpdump / Wireshark.
 - LS-LI blackjack/data 0 הרשאות הסלודא PEM-על קובץ ה 600סלודא.

2.6 תיאור פרוטוקול התקשורת

2.6.1 תיאור מילולי

הפרוטוקול עובד בשלוש שכבות, מהנמוכה לגבוהה:

- שכבת המסגור (framing): כל הודעה נשלחת כך - 4 בתים big-endian unsigned int. הצד המקבל קורא תחילה 4 בתים, מפענח את האורך, מוודא $0 < n \leq 65,536$ וקורא n בתים.
- שכבת ההצפנה: עד לסיום ה-handshake הגוף הוא JSON גלוי. לאחר מכן, הגוף הוא פלט של HMAC) טוקן שכולל גם בקרה על שלמות - Fernet.encrypt(plaintext) (בתוך ה-Fernet) הוא JSON עם המבנה {"type": "VALID_TYPES", "data": {...}}. סוג ההודעה חייב להיות אחד מתוך רשימת {"...", "data": {...}}. הסגורה.

מגבלת אבטחה: MAX_MESSAGE_BYTES = 64 KB - מסגרת גדולה יותר נדחית מיידית, החיבור נסגר.

2.6.2 פירוט ההודעות

שם הודעה	נשלח מ→אל	שדות בגוף ה-Hודעה
server_hello	שרת → לקוח	public_key_pem: str
key_exchange	לקוח → שרת	encrypted_session_key: str (base64)
ready	שרת → לקוח	ריק; משדר שההצפנה הסימטרית פועלת
register	לקוח → שרת	username: str, password: str
login	לקוח → שרת	username: str, password: str
auth_result	שרת → לקוח	bool, message: str, profile?: dic
profile	שרת → לקוח	profile: dict (username, credits, win)
place_bet	לקוח → שרת	amount: int (50..5000)
action	לקוח → שרת	action: "hit"\

"stand"

"double"

game_state	phase: str, dealer: dict, players: li	
round_result	שרת → לקוח	outcome: "win" \ "loss", payout: int, profile: dict
chat	sender: str, message: str (≤200 תווים)	
error	שרת → לקוח	message: str
logout	לקוח → שרת	{}

2.7 תיאור מסכי המערכת

2.7.1 מסך התחברות / הרשמה

- תפקיד: קליטת אישורי גישה, שליחתם לשרת, הצגת תשובת השרת.
- תיאור: חלון 400×260, רקע ירוק כהה. שדה Username, שדה Password (תווים מוסתרים), שני כפתורים - Login (זהב) ו-Register. תווית מצב (Status) למטה להודעות שגיאה / הצלחה.
- תמונת מסך: (להוסיף בשלב הסופי - צילום של LoginFrame עם username = alice ושגיאת התחברות)

2.7.2 מסך השולחן

- תפקיד: ההתרחשות העיקרית - הימור, פעולות, צפייה בקלפים.
- תיאור: סרגל פרופיל עליון Player: alice Credits: 9,750 W: 4 L: 2 W/L: 2.00), אזור דילר באמצע (כותרת + שורת קלפים), אזור שחקנים מתחתיו (כל שחקן בשורה, משלו, עם הדגשה זהובה למשתמש עצמו), שדה הימור עם כפתור Place Bet, שורת כפתורים וצי'אט בצד ימין Hit / Stand / Double.
- תמונת מסך: (להוסיף בשלב הסופי - צילום של TableFrame עם 3-2 שחקנים בתוך סבב פעיל)

2.7.3 דיאלוג תוצאת סבב

- תפקיד: להציג בולטות את תוצאת הסבב לשחקן.
- תיאור: באנר צבעוני מעל אזור השחקנים (זהב = WIN, אדום = LOSS, אפור = PUSH) למשך זמן ההפסקה (6 שניות) ואחר כך מוסר עם תחילת סבב חדש.

2.7.4 תרשים מסכים (Screen Flow)

```

+-----+ login OK +-----+
| LoginFrame |----->| TableFrame |
+-----+ +-----+
      ^         ^
      |         |
      |         | disconnect / logout
      |         |
      |         |
      |         |
      |         | register OK (auto-login)
      |         |
      +-----+

```

2.8 תיאור מבני הנתונים

2.8.1 קובץ blackjack/data/profiles.json

- שם המאגר: profiles (מילון שטוח ב-JSON).
- המפתח הראשי: username (מחרוזת).
- שדות ברשומת משתמש:

דוגמה	אורך / טווח	טיפוס	שם השדה
"alice"	16-3 תווים, [A-Za-z0-9_]	str	username
"k4ja...\$g2Cv..."	פורמט "\$<salt_b64>\$<hash_b64>"	str	password_hash
9750	≥ 0	int	credits
4	≥ 0	int	wins
2	≥ 0	int	losses
1	≥ 0	int	pushes

- שמירה אוטומית: הקובץ נכתב למקום זמני profiles.json.tmp ואז os.replace – פעולה אוטומטית במערכות קבצים מודרניות.

2.8.2 קובץ blackjack/data/server_rsa.pem

- תוכן: מפתח RSA פרטי בפורמט PEM.
- הרשאות: chmod 0o600 – קריאה / כתיבה רק עבור בעל הקובץ. נוצר אוטומטית בריצה הראשונה.

2.8.3 מבני נתונים בזיכרון

- ProfileManager.profiles : Dict[str, UserProfile] – מילון שמאחסן את כל הפרופילים בזיכרון; משוקף לדיסק על כל שינוי.
- BlackjackServer._clients : List[ClientHandler] – רשימת המטפלים הפעילים.
- BlackjackServer._online : Dict[str, ClientHandler] – מיון משם משתמש למטפל הפעיל שלו (למניעת התחברות כפולה).
- Table._players : List[Player] – רשימת שחקנים בסדר ישיבה.
 - Deck._cards : List[Card] – חבילת הקלפים הפנימית.
- AppController._queue : queue.Queue – תור הודעות מהרשת אל ה-GUI.

2.9 סקירת חולשות ואיומים

2.9.1 שכבת האפליקציה

פתרון בפרויקט	האם רלוונטי?	איום
-	לא רלוונטי – אין DB יחסית, אין מחרוזות	**SQL Injection**
סיסמאות מגובבות עם PBKDF2-HMAC-SHA256 רלוונטי		**תהליך login (אימות)**
הצפנה היברידית: לחיצת ID RSA-2048 + OA	רלוונטי – זה לב הפרויקט	**MITM (Man-in-the-Middle)**

DoS / DDoS	רלוונטי בעיקר ל-DoS על שרת בודד	מגבלת גודל הודעה 64 KB; דחיית הודעה
fuzzing / פרוטוקול שגוי	רלוונטי	חי - _decode_message - קפדני ב parser
העלאת קבצים	לא רלוונטי - אין העלאות	-
עבודה עם אתרי web	לא רלוונטי - יישום שולחני	-
גישה לנתוני משתמש אחר	רלוונטי	כל בקשה דורשת require_auth; פעולה ב
חולשה ב-randomness	רלוונטי לקליפים	שימוש ב-secrets (/dev/urandom) ולא
שמירת סיסמה בזיכרון / לוג	רלוונטי	סיסמאות אינן נכתבות ל-log אף פעם; אין

2.9.2 שכבת התעבורה

- מבטיחה. socket.connect-לחיצת יד משולשת: מתבצעת ע"י מערכת ההפעלה כחלק מ - TCP חיבור אמין ומסונכרן.
- הצפנה ברמת התעבורה: הפרויקט אינו משתמש ב-TLS (אין לנו תעודה רשמית); במקום זאת, בנינו hybrid handshake משלנו, שמספק את אותן ערבויות סודיות + שלמות למשך החיבור. כפיית סדר הודעות: TCP מבטיח סדר; בנוסף, recv_exact דואג שלא נקרא חצי הודעה.

2.9.3 הפעלת המערכת

- חולשות אפשריות במהלך ההרצה:
 - אם משתמש מקבל גישה למחשב השרת, יוכל לקרוא את profiles.json - אך הסיסמאות - מגובבות ולכן לא חשופות. server_rsa.pem מוגן 0o600.
 - אם משתמש מקבל גישה למחשב הלקוח - יוכל לראות את הקרדיטים שלו (אך לא של אחרים, - כי לא מאוחסנים שם).
 - טופלו: הרשאות קובץ ה-PEM, אי-כתיבת סיסמאות בלוג, מניעת login כפול, גילוי נסיונות אינ'קציה דרך parser קפדני.

3. מימוש הפרויקט

3.1 חלק א' - מודולים ומחלקות

3.1.1 מודולים מיובאים (חיצוניים / סטנדרטיים)

שם המודול	למה מיועד
socket	תקשורת TCP גולמית (יצירה / הקשבה / קבל
threading	תהליכון לכל לקוח מחובר ולתור הסבב הבא
struct	קידוד אורך הודעה כ-4 בתים big-endian
json	קידוד / פענוח של גוף ההודעה
secrets	מקור אקראיות קריפטוגרפי לערבוב הקלפים
hmac, hashlib	constant-time והשוואה PBKDF2
base64	קידוד מפתחות בינאריים בתוך JSON
dataclasses	הגדרה תמציתית של Card, UserProfile
enum	מצבי המשחק - Phase
logging	רישום לוגים בצד השרת
queue	תור הודעות בין thread הרשת ל-GUI
tkinter, tkinter.ttk, tkinter.mes	ממשק משתמש גרפי
cryptography.hazmat.primitives.asymme	RSA-2048 + OAEP
cryptography.hazmat.primitives.serial	פורמט PEM
cryptography.fernet.Fernet	הצפנה סימטרית מאומתת

3.1.2 מחלקות שפותחו במסגרת הפרויקט

“ פירוט מלא של כל הפעולות נמצא ב-flowchart_deep.html. כאן מובא טבלה תמציתית.

◆ Card (common/card.py)

- תפקיד: קלף בודד, immutable.
- תכונות: (rank: str ('A','2'..'K'), suit: str ('S','H','D','C')).
- פעולות: __post_init__ - ולידציה; base_value - ערך בסיסי (אס=1, פנים=10); is_ace; to_dict / from_dict - סריאליזציה.

◆ Deck (common/deck.py)

- תפקיד: "shoe" של N חבילות (ברירת מחדל 4) מעורבבות יחד.
- תכונות: _cards: list[Card], num_decks: int.
- פעולות: Fisher-Yates - reshuffle() על Card -> draw() secrets.randbelow; - שליפה ועירבוב מחדש כשהמלאי >15; __len__.

◆ Hand (common/hand.py)

- תפקיד: יד של שחקן או דילר עם חישוב ערך חכם של אסים.
- תכונות: _cards: list[Card].

- פעולות: `add(card); clear(); value (property); is_soft; is_blackjack`; `is_bust; to_list()`.

◆ **SecureChannel (common/crypto.py)**

- תפקיד: עטיפה ל-Fernet שמייצגת ערוץ סימטרי מאובטח.
- תכונות: `._fernet: Fernet`.
- פעולות: `generate() (classmethod); encrypt(plaintext); decrypt(token)`.

◆ **RSAPair (common/crypto.py)**

- תפקיד: מפתחות RSA-2048 של השרת.
- תכונות: `private_key, public_key`.
- פעולות: `generate(); load_or_create(path); public_pem()`; `decrypt_session_key(b64)`.

◆ **UserProfile (server/profile_manager.py)**

- תפקיד: רשומת משתמש בודדת.
- תכונות: `username, password_hash, credits, wins, losses, pushes`.
- פעולות: `games_played (property); wl_ratio (property); to_public_dict()` - ללא `hash`.

◆ **ProfileManager (server/profile_manager.py)**

- תפקיד: טוען / שומר את `profiles.json`, מספק רישום, אימות, שינוי קרדיטים ועדכון תוצאות.
- תכונות: `profiles: dict, _lock: RLock, path: str`.
- פעולות: `_load(); _save_locked(); validate_username/password()` (סטטיים); `register(u,p); authenticate(u,p); adjust_credits(u, delta); record_result(u, outcome); get(u)`.

◆ **Phase (server/game.py)**

- תפקיד: `enum` של חמשת השלבים - `WAITING, BETTING, PLAYING, DEALER, RESULTS`.

◆ **Participant, Dealer, Player (server/game.py)**

- תפקיד: היררכיית ירושה - `Participant` (שם + יד), `Dealer` (כלל `should_hit`), `Player` (יתרת הימור, סטטוסים).
- תכונות עיקריות של `Player`: `username, bet, has_bet, finished, doubled, outcome, payout`.
- פעולות: `reset()`.

◆ **Table (server/game.py)**

- תפקיד: מכונת מצבים של שולחן בלאק-ג'יק.
- תכונות: `_players, _dealer, _deck, _phase, _current_idx, _lock`; `on_round_finished` ו-`callbacks_broadcast` `RLock`.
- פעולות: `add_player, remove_player, place_bet, player_action`; `_start_round_locked, _run_dealer_locked, _resolve_locked, prepare_next_round, snapshot_for(username)`.

◆ **ClientHandler (server/client_handler.py) - threading.Thread יורש מ**

- תפקיד: `thread` אחד לכל לקוח מחובר.

- תכונות: `_sock`, `_channel`: `SecureChannel`, `_profile`: `UserProfile` | `None`
`_send_lock`.
- פעולות: `run()`; `_handshake()`; `_main_loop()`; `_dispatch(type,data)`
`_handle_register / _handle_login`; `send(type,data)`; `shutdown()`.

◆ **BlackjackServer (server/server.py)**

- תפקיד: המנצח הראשי - `listening socket`, `accept loop`, `broadcasts`
 תכונות: `_sock`, `_clients`: `list`, `_online`: `dict`, `_rsa`: `RSAPublicKey`, `_profiles`: `ProfileManager`, `_table`: `Table`, `_round_timer`: `Timer` | `None`.
 פעולות: `serve_forever()`, `stop()`, `on_client_authenticated`
`on_client_disconnected`, `handle_place_bet`, `handle_action`, `handle_chat`,
`_send_profile`, `_broadcast_state`, `_handle_round_finished`,
`_start_next_round`.

◆ **BlackjackClient (client/network.py)**

- תפקיד: שכבת רשת בלקוח, כולל `thread` קבלה.
- תכונות: `_sock`, `_channel`: `SecureChannel`, `_recv_thread`, `_send_lock`
`callbacks on_message / on_disconnect`.
- פעולות: `connect()`, `_handshake()`, `send(type,data)`, `_recv_loop()`
`close()`.

◆ **LoginFrame, TableFrame, AppController (client/gui.py)**

- תפקידים: מסך התחברות, מסך השולחן, ובקר ראשי שמחבר ביניהם ובין הרשת.
- פעולות עיקריות של `AppController`: `run()`, `send(type,data)`, `do_login`
`do_register`, `do_logout`, `_enqueue_message`, `_drain_messages`,
`_handle_message`, `_show_login / _show_table`, `_on_close`.

3.2 חלק ב' - קטעי קוד מרכזיים

3.2.1 חישוב ערך יד עם אסים

```
# blackjack/common/hand.py
@property
def value(self) -> int:
    """Best-possible Blackjack value of this hand.

    All aces start as 1; we then 'upgrade' aces to 11 one at a time
    as long as it does not push the total over 21. This handles all
    legal hand combinations in O(n).
    """
    total = sum(c.base_value for c in self._cards) # all aces = 1
    aces = sum(1 for c in self._cards if c.is_ace)
    while aces > 0 and total + 10 <= 21:
        total += 10 # promote one ace from 1 to 11
        aces -= 1
    return total
```

3.2.2 ערבוב מאובטח של החבילה

```
# blackjack/common/deck.py
def reshuffle(self) -> None:
    """Build a fresh shoe and shuffle it using a cryptographic RNG."""
    self._cards = [
        Card(rank, suit)
        for _ in range(self.num_decks)
        for suit in Card.SUITS
        for rank in Card.RANKS
    ]
    # Fisher-Yates with secrets.randbelow → unpredictable order
    for i in range(len(self._cards) - 1, 0, -1):
        j = secrets.randbelow(i + 1)
        self._cards[i], self._cards[j] = self._cards[j], self._cards[i]
```

3.2.3 לחיצת יד הצפנה (צד שרת)

```
# blackjack/server/client_handler.py
def _handshake(self) -> None:
    # 1.
    send_plain(self._sock, "server_hello",
               {"public_key_pem": self._rsa.public_pem()})
    # 2.    Fernet -RSA
    msg = recv_plain(self._sock)
    if msg["type"] != "key_exchange":
        raise ProtocolError("expected key_exchange")
    session_key = self._rsa.decrypt_session_key(
        msg["data"]["encrypted_session_key"])
    # 3.    -Fernet
    self._channel = SecureChannel(session_key)
    # 'ready' -
    send_encrypted(self._sock, self._channel, "ready", {})
```

3.2.4 גיבוב והשוואת סיסמאות בזמן קבוע

```
# blackjack/common/crypto.py
ITERATIONS = 200_000

def hash_password(password: str) -> str:
    salt = secrets.token_bytes(16)
    digest = hashlib.pbkdf2_hmac(
        "sha256", password.encode("utf-8"), salt, ITERATIONS)
    return f"{base64.b64encode(salt).decode()}$" \
        f"{base64.b64encode(digest).decode()}"

def verify_password(password: str, stored: str) -> bool:
    try:
        salt_b64, hash_b64 = stored.split("$", 1)
        salt = base64.b64decode(salt_b64)
        expected = base64.b64decode(hash_b64)
    except (ValueError, binascii.Error):
        return False
    actual = hashlib.pbkdf2_hmac(
        "sha256", password.encode("utf-8"), salt, ITERATIONS)
    # constant-time comparison – timing attacks
    return hmac.compare_digest(actual, expected)
```

3.2.5 מסגור הודעה עם הגנה על גודל

```
# blackjack/common/protocol.py
MAX_MESSAGE_BYTES = 64 * 1024
_LENGTH_PREFIX = struct.Struct(">I") # 4-byte big-endian unsigned int

def send_frame(sock: socket.socket, payload: bytes) -> None:
    if len(payload) > MAX_MESSAGE_BYTES:
        raise ProtocolError("payload too large")
    sock.sendall(_LENGTH_PREFIX.pack(len(payload)) + payload)

def recv_frame(sock: socket.socket) -> bytes:
    header = _recv_exact(sock, _LENGTH_PREFIX.size)
    (length,) = _LENGTH_PREFIX.unpack(header)
    if length == 0 or length > MAX_MESSAGE_BYTES:
        raise ProtocolError(f"invalid frame length: {length}")
    return _recv_exact(sock, length)
```

3.2.6 צילום מצב בטוח לכל שחקן (הסתרת קלף הדילר)

```
# blackjack/server/game.py
def snapshot_for(self, username: str) -> dict:
    with self._lock:
        dealer_cards = self._dealer.hand.to_list()
        # -
        if self._phase is Phase.PLAYING and len(dealer_cards) >= 2:
            dealer_cards = [dealer_cards[0],
                            {"rank": "?", "suit": "?"}]
            value_hidden = True
        else:
            value_hidden = False
    return {
        "phase": self._phase.value,
        "dealer": {"cards": dealer_cards,
                  "value_hidden": value_hidden,
                  "value": self._dealer.hand.value
                  if not value_hidden else None},
        "players": [self._player_dict(p, p.username == username)
                    for p in self._players],
        "current_username": (self.current_player().username
                             if self.current_player() else None),
        "deck_remaining": len(self._deck),
    }
```

3.3 חלק ג' - מסמך בדיקות

3.3.1 בדיקות שתוכננו בשלב האפיון

בעיות → פתרון	תוצאות	מה בוצע בפועל	מטרה	#
-	cr קיבל רשומה חדשה עם json, הפרופיל	alice / secret123 הזנתי	אליס הקינה	1
-	התקבלה הודעת "already taken"	אליס המשיכה לנסות להירשם	מניעת רישום כפול	2
-	בגרסה הראשונה הופיעה הודעת "Invalid message" (ok:false, "message")	אליס הזנתה alice / wrong password	סיסמה שגויה	3
-	חיבור א' ל "User already logged in"	אליס ניסתה להיכנס שוב	ניסיון להיכנס שוב	4
-	בתחילה התשלום על בלאק-ג'יה היה 1000, לאחר מכן 200/300	אליס ניסתה להיכנס עם 1000	החלפת מחיר	5
-	היתרה: "Insufficient credits"	אליס ניסתה להיכנס עם 1000	היתרה	6
-	השרת השיב "Not your turn"	אליס ניסתה להיכנס עם 1000	היתרה	7
-	החבילות לפני ה-handshake הכוללות 5050X	אליס ניסתה להיכנס עם 1000	היתרה	8
-	בהתחלה השרת ניסה להקצות 100KB, לאחר מכן 200KB	אליס ניסתה להיכנס עם 1000	היתרה	9
-	קרדיטים, ניצחונות והפסדים נשמרים	אליס ניסתה להיכנס עם 1000	היתרה	10
-	בתחילה השרת תקע על "מחכה תשחק"	אליס ניסתה להיכנס עם 1000	היתרה	11
-	נדחה כבר ב-GUI עם "may only use name"	אליס ניסתה להיכנס עם 1000	היתרה	12

3.3.2 בדיקות נוספות שבוצעו

בעיות → פתרון	תוצאה	מה בוצע	מטרה	#
-	עבר לחלוטין	הרצת blackjack/test_e2e.py	בדיקת סיום	13

14	jack/data/server_rsa.pem	האובייקט PEM קובץ	בכתיבה הראשונית הפרמיסיות (044000) הם -rw---
-	SecureChannel-ProtocolError;	החיבור נסגר; שאר השחקן	ההבדל בין handshake ו-ProtocolError
-	handshake	כולם עברו את ה-handshake	ההבדל בין handshake ו-ProtocolError
17	secrets vs random	אין דפוס ניתן לחיזוי; secrets	ההבדל בין secrets ו-random
18	connection lost	הלקוח מציג דיאלוג "connection lost"	ההבדל בין connection lost ו-ProtocolError

4. מדריך למשתמש

4.1 פירוט קבצי המערכת - עץ קבצים

```

blackjack/
├── README.md
├── .md
├── flowchart.html
├── flowchart_deep.html
├── .gitignore
├── run_server.py
├── run_client.py
├── common/
│   ├── __init__.py
│   ├── card.py
│   ├── deck.py
│   ├── hand.py
│   ├── protocol.py
│   └── crypto.py
├── server/
│   ├── __init__.py
│   ├── profile_manager.py
│   ├── game.py
│   ├── client_handler.py
│   └── server.py
├── client/
│   ├── __init__.py
│   ├── network.py
│   └── gui.py
└── data/
    ├── profiles.json
    └── server_rsa.pem

```

← high-level

← -data/ -__pycache__/

← Card

← Deck

← Hand

← framing + JSON envelope +

← RSAKeyPair, SecureChannel, hash_password

← UserProfile + ProfileManager

← Phase, Participant, Player, Dealer, Table

← Thread

← BlackjackServer (accept loop + broadcasts)

← BlackjackClient

← LoginFrame + TableFrame + AppController

← (gitignored)

4.2 התקנת המערכת

4.2.1 הסביבה הנדרשת

- (ומעלה) מומלץ Python 3.10 3.11
- מערכת הפעלה: Linux / macOS / Windows
- פעיל pip
- רשת מקומית בה השרת והלקוחות יכולים לראות זה את זה (או אותו מחשב).

4.2.2 הכלים הנדרשים

- התקנה של החבילה החיצונית היחידה:

```
`bash
```

```
pip install cryptography
```

```
`
```


- ייתכן שיש להתקין Python (ב-Linux) כבר כלול בהתקנה הסטנדרטית של Tkinter (python3-tk).

4.2.3 מיקומי קבצים

- יש לפתוח את תיקיית הפרויקט (זו שמכילה את התיקייה blackjack) במסוף.
 - בריצה ראשונה ייווצר אוטומטית: blackjack/data/profiles.json, blackjack/data/server_rsa.pem.

4.2.4 נתונים התחלתיים

- אין משתמש ברירת מחדל. כל משתמש יוצר חשבון בעצמו דרך מסך ההרשמה.
 - קרדיטים התחלתיים לכל משתמש חדש: 10,000.
 - גבולות הימור: $50 \leq \text{bet} \leq 5000$.

4.2.5 רשת

- ברירת מחדל: שרת מאזין על 0.0.0.0:5050; לקוח מתחבר ל-127.0.0.1:5050.
 - לשינוי, יש להגדיר משתני סביבה לפני ההפעלה:

```
`bash
```

```
BJ_HOST=192.168.1.10 BJ_PORT=5050 python -m blackjack.run_client
```

```
`
```

4.2.6 ארכיטקטורה מינימלית נדרשת

- מחשב יחיד 1 GB RAM, 512 MB, GHz, מקום פנוי ≥ 5 MB. מערכת המשחק קלה במיוחד.

4.3 הפעלת המערכת

4.3.1 הפעלת השרת

```
python -m blackjack.run_server
```

יודפס בלוג מסר כגון:

```
INFO Server listening on 0.0.0.0:5050
```

לעצירה: Ctrl+C (השרת סוגר בנימוס את כל החיבורים).

4.3.2 הפעלת הלקוח

על אותו מחשב או על מחשב אחר באותה רשת:

```
python -m blackjack.run_client
```

נפתח חלון התחברות. המשתמש יזין שם משתמש וסיסמה ויקליק Register (פעם ראשונה) או Login (פעמים הבאות).

4.3.3 צילומי מסך עם הסברים בשפה פשוטה

“ (להוסיף בגרסה הסופית לאחר הצילום בפועל. בכל צילום מסך לכלול חץ או הסבר קצר במשפט.)

- צילום 1 – מסך התחברות: "כאן מקלידים שם משתמש וסיסמה. אם לא נרשמת קודם, לחץ על Register."
- צילום 2 – מסך השולחן בתחילת סבב: "בסרגל העליון רואים את הקרדיטים והסטטיסטיקה שלך. הקלד את ההימור ולחץ Place Bet."
- צילום 3 – אמצע סבב, התור שלך: "הקלפים שלך מוצגים בשורה שלך. לחץ Hit לקלף נוסף, לעצירה Stand."
- צילום 4 – הדילר משחק: "הקלף הנעלם נחשף ועוד קלפים מתווספים אוטומטית."
- צילום 5 – תוצאה: "באנר זהוב = ניצחת; אדום = הפסדת; אפור = פוש (תיקו)."

5. סיכום אישי / רפלקציה

עבודה על הפרויקט הזה הייתה עבורי מסע ארוך אך מעורר השראה. בתחילה היה לי רעיון מאוד מעורפל - "אעשה משחק קלפים עם רשת" - אבל מהר מאוד גיליתי שכל מילה בתיאור הזה מסתירה עולם שלם של החלטות. במהלך הפיתוח חוויתי שלוש "הצלחות גדולות" ושני "תקיעות עיקריות" שעיצבו את ההבנה שלי בנושא.

הצלחות: הראשונה הייתה ברגע שהפעלתי שני לקוחות בו-זמנית והם באמת ראו אותו שולחן בלאק-ג'ק. עד הרגע הזה ה-threads והרשת היו עבורי מילים בספר; אחר כך הם הפכו לכלי עבודה. ההצלחה השנייה הייתה כש-tcpdump הראה לי שאחרי ה-handshake התעבורה אכן לא קריאה - ההצפנה לא הייתה רק תיאוריה. השלישית הייתה כשהבנתי לבד למה random אינו מתאים למשחק קזינו, וחיפשתי בעצמי את secrets והחלפתי הכל.

אתגרים וקשיים: הקושי הגדול הראשון היה הבנת ההבדל בין "ביט" לבין "מחרוזת" ב-Python 3 - שעות שלמות הוצאו על שגיאות על שגיאות `TypeError: a bytes-like object is required, not 'str'`. הקושי השני היה `race conditions`: בגרסה ראשונה לפעמים לא היה אפשר להמר אחרי שמישהו אחר שלח Hit באותו רגע. רק אחרי שלמדתי על `threading.RLock` והתחלתי לעטוף כל פעולה משנה-מצב הפסיק זה לקרות.

תהליך הלמידה: רוב הלמידה הייתה עצמאית. ה-MDN ו-Real Python העניקו לי הקדמה ל-sockets, התיעוד הרשמי של cryptography העניק לי את החלקים של RSA + Fernet, וסרטוני YouTube של עזרו לי להבין מבחינה אינטואיטיבית את ההבדל בין הצפנה אסימטרית לסימטרית Computerphile. בכל פעם שנתקעתי כתבתי לעצמי "מה אני בדיוק מנסה להעביר ולמה" - הטקסט הזה לבדו פתר חצי מהבאגים.

כלים שנלקחים להמשך: העבודה לימדה אותי שלוש הרגלים שאקח איתי לכל פרויקט עתידי: (1) להגדיר פרוטוקול תקשורת בכתב לפני שכותבים שורת קוד; (2) להשתמש בלוגינג מסודר במקום כך שכל קובץ ניתן לקריאה לבדו - `common, server, client` - להפריד שכבות (3) `print`;

תובנות: השיתוף עם המנחה והעמיתים היה קריטי. בכל פעם שהראיתי למישהו את הקוד הוא שאל "ומה קורה אם..." וזה גרם לי לחשוב על מקרי קצה שלא חשבתי עליהם. למדתי שלמידת עמיתים לא נועדה רק כדי לקבל עזרה אלא כדי שתהיה לך סיבה להסביר לעצמך את מה שאתה כותב.

במבט לאחור - חלקים שהייתי משנה: הייתי בוחר במנגנון אסינכרוני (`asyncio`) במקום הייתי גם משתמש `locks`. כי ככל שהוספתי לקוחות יותר נוצרה חווית "עומס" של `threading`, מההתחלה בספריית `pytest` עם בדיקות יחידה, ולא רק בדיקות אינטגרציה ידניות.

אם היו לי משאבים נוספים הייתי מוסיף: (1) תמיכה בכמה שולחנות במקביל; (2) `WebSocket` handshake אמיתית במקום TLS כדי שלקוחות בדפדפן יוכלו להצטרף; (3) תמיכה בתעודת `bridge` ביתי; (4) בדיקות יחידה אוטומטיות עם `CI`; (5) אנימציות חלוקה של קלפים ב-Tkinter.

שאלות חקר שאני עדיין נושא איתי: האם `secrets.randbelow` באמת בלתי ניתן לחיזוי לחלוטין, או שיש מתקפות על מקור `urandom`? כיצד באמת עובד OAEP "מתחת למכסה המנוע"? איך קזינואים אמיתיים מאמתים שהדילר אכן נטל קלף מצמרת חבילה ולא מהאמצע? כל אחת מהשאלות הללו תפתח לי דלת לפרויקט הבא.

תודות - למורה ולמנחה שהשקיעו זמן בקריאת הקוד וביציאת הצ'ק-אינים השבועיים; למשפחתי על הסבלנות בכל פעם ש"רק עוד באג קטן" הפך לחצי לילה; ולקהילה הפתוחה של Python על תיעוד שאי-אפשר להפריז בערכו.

6. ביבליוגרפיה

(בכתיב APA-7)

- Python Software Foundation. (2024). socket — Low-level networking interface. Retrieved from <https://docs.python.org/3/library/socket.html>
- Python Software Foundation. (2024). threading — Thread-based parallelism. Retrieved from <https://docs.python.org/3/library/threading.html>
- Python Software Foundation. (2024). secrets — Generate secure random numbers for managing secrets. Retrieved from <https://docs.python.org/3/library/secrets.html>
- Python Software Foundation. (2024). hashlib — Secure hashes and message digests. Retrieved from <https://docs.python.org/3/library/hashlib.html>
- Python Cryptographic Authority. (2024). Cryptography documentation. Retrieved from <https://cryptography.io/en/latest/>
- Bellare, M., & Rogaway, P. (1995). Optimal asymmetric encryption — How to encrypt with RSA. EUROCRYPT '94, LNCS 950.
- OWASP Foundation. (2023). Password Storage Cheat Sheet. Retrieved from https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- Stevens, W. R., & Rago, S. A. (2013). Advanced Programming in the UNIX Environment (3rd ed.). Addison-Wesley.
- Tanenbaum, A. S., & Wetherall, D. J. (2011). Computer Networks (5th ed.). Pearson.
- Computerphile. (2017). Public Key Cryptography [Video]. YouTube. https://www.youtube.com/watch?v=GSIDS_lvRv4

7. נספחים

נספח א' - פלואו צ'ארטים אינטראקטיביים

- blackjack/flowchart.html - סיפור (13 שלבים, ספור - הסב).
- blackjack/flowchart_deep.html - (90 פריטים ~) מחלקה ופונקציה (ציליל לכל קובץ, מחלקה ופונקציה).

נספח ב' - הסבר על הטכנולוגיות בהן נעשה שימוש

B.1 RSA-2048 + OAEP

הוא אלגוריתם הצפנה אסימטרי - יש בו זוג מפתחות, ציבורי ופרטי. כל מה שמוצפן עם RSA הציבורי ניתן לפיענוח רק עם הפרטי. אנחנו משתמשים בו פעם אחת בכל חיבור, להעברת מפתח שמוסיף אקראיות לפני ההצפנה, ובכך מונע מתקפות padding הוא מנגנון OAEP. סימטרי Fernet על RSA "נקי".

B.2 Fernet

authenticated האריזה שלה נקראת AES-128 CBC ו-HMAC-SHA256 היא מפרט המבוסס על Fernet שמאפשר timestamp כל ניסיון לשנות בית בודד יזהה במהלך הפענוח. בנוסף נכלל: encryption: זיהוי הודעות "שנות".

B.3 PBKDF2

(salt) על הסיסמה והמלח HMAC הפעלה חוזרת של Password-Based Key Derivation Function: 200,000 איטרציות הופכות כל ניסיון ניחוש ליקר חישובית, מה שהופך מתקפת brute-force להרבה פחות כדאית.

B.4 TCP

מקנה זרם בתים אמין ומסונכרן בין שני קצוות. מערכת - Transmission Control Protocol ההפעלה מנהלת את החיבור (לחיצת יד משולשת SYN/SYN-ACK/ACK), מספור חבילות, ושליחה חוזרת בכשל. אנחנו בונים מעליו פרוטוקול הודעות משלנו (length-prefix).

B.5 Threads ו-Locks

המנסים לקרוא threads הוא "תהליכון" - נתיב ביצוע נוסף בתוך אותה תוכנה. שני Thread ולכתוב לאותו משתנה בו זמנית עלולים להגיע ל-Lock condition. (וביתר ספציפית אחד "מחזיק את המפתח" באותו רגע, וכל אחר ימתין thread מבטיח שרק RLock)

נספח ג' - הדפסת קוד מלאה

“ בגרסה הסופית להדפסה יש לצרף בנספח זה את כל הקוד מהקבצים הבאים, בסדר הקריאה המומלץ. כל קובץ כולל הערות (docstrings) ב

- blackjack/common/card.py
- blackjack/common/deck.py
- blackjack/common/hand.py
- blackjack/common/protocol.py
- blackjack/common/crypto.py
- blackjack/server/profile_manager.py
 - blackjack/server/game.py
- blackjack/server/client_handler.py
 - blackjack/server/server.py
- blackjack/client/network.py
 - blackjack/client/gui.py
 - blackjack/run_server.py
 - blackjack/run_client.py

(הקוד המלא נמצא בתיקיית הפרויקט ומוכן להדפסה ישירות מכל IDE.)